Final Report

UNZA Hostel Maintenance Request and Tracking System

University of Zambia — Final Year Project

Final Report

Demand & Final Submission deliverable (UP): a reflection on how the Unified Process actually played out across this project's 15 weeks, what was learned, and a traceability summary tying the original vision back to what was actually delivered, tested and documented. For the live-demo script, see VIVA_GUIDE.md §3; for the full artefact set this report draws on, see the document list in README.md.

---

1. Project summary

The **UNZA Hostel Maintenance Request and Tracking System** replaces verbal/paper fault reporting in university hostels with a Progressive Web App covering the full lifecycle — `Open → Assigned → In Progress → Resolved → Closed` — across three roles (student, technician, administrator), on a client-server architecture with a layered server (routes → services → repositories) and a Supabase (PostgreSQL) data tier. Beyond the brief's core requirements, the delivered system also includes upvoting/duplicate detection, SLA timers with overdue alerts, live updates, AI-assisted triage, camera-capture proof of work, dark mode, analytics with a hostel heatmap, a weekly emailed summary, hostel-targeted announcements, per-request comments, recurring-issue detection, technician performance tracking, preventive-maintenance scheduling, and an in-app feedback channel — see VIVA_GUIDE.md §4 for the brief-by-brief mapping.

2. How the Unified Process actually played out

UP idea	How it held up in practice
Risk-driven: tackle the riskiest, least-understood parts first	Held. The architecture's central bet — the client-server, three-layer design — was proven in Elaboration Iteration 1 on the core lifecycle (register/login/submit/assign/update-status) before any peripheral feature existed. Every one of the ~30 features added afterward (comments, announcements, preventive maintenance, app feedback, ...) reused that same proven shape without needing to revisit it — see LOGICAL_ARCHITECTURE.md.
Use-case driven	Held, with a deliberate discipline about depth: 12 of ~40 use cases were fully dressed (the ones that actually carried risk or a non-obvious rule); the rest were captured casually once the pattern was proven — see USE_CASES.md §5. Writing every one of the ~40 to the same depth as the primary 12 would not have found more risk, only spent more time restating an already-proven pattern.
Architecture-centric	Held. The layering rule (client never talks to Supabase; routes → services → repositories, only calling downward) was fixed in Elaboration and never violated afterward — not because it was never tempting to shortcut under time pressure, but because it was cheap to check (a route with a SQL query in it, or a service that imports Express types, is immediately visible in review) — see R1 in RISK_REGISTER.md.

UP idea	How it held up in practice
Iterative and incremental	Held, and this is the project's clearest practical lesson (§4). Construction did not proceed as one long build phase — features shipped as small, independently committed, independently tested increments, right through to the very last week, which is *why* an 11-feature batch requested in a single late session (comments, announcements, recurring-issue detection, technician performance, preventive maintenance, and more, in one sitting) was survivable at all: each one was still its own reviewable, revertible commit, not one undifferentiated mass of change.

3. Requirements traceability (summary)

From the brief	Delivered in	Verified by
Registration, ID-based login, roles	UC-1, UC-2	`auth.service.test.js`, integration auth tests
Submit a request (category, description, photo)	UC-3, UC-9	`requests.service.test.js`, integration submission tests
Lifecycle enforcement	UC-16	`requests.service.test.js` (`isValidTransition`), integration lifecycle tests
Administrator assignment	UC-23	`users.service.test.js` (ranking), integration assignment tests
Technician status updates + proof of work	UC-16, UC-17	integration lifecycle + proof-of-work tests
Student real-time tracking	UC-4, UC-5	`events.service.test.js`, manual UAT scenario 5/7 (BETA_TESTING_REPORT.md)
Status history	all lifecycle use cases	integration tests asserting a `status_history` row on every transition
Analytics	UC-27, UC-30, UC-31	`reports.service.test.js`, `performance.service.test.js`, `recurrence.service.test.js`
Role-based access control	every use case	8 dedicated integration test cases + RLS (TECHNICAL_DOCUMENTATION.md §5.3)
PWA: offline, installable, push	UC-10	manual UAT scenarios 18–20
Beyond the brief (10 features)	UC-6 through UC-40	see VIVA_GUIDE.md §4 for the full list

Every row above traces from a **use case** (what a stakeholder asked for) to **code** (what was built) to a **test** (how it is known to still work) — the chain a viva question can be answered by walking, in either direction.

4. Lessons learned

Technical

- **A pure-function business rule, written once and imported everywhere it applies, is worth the small extra step of factoring it out.** `utils/sla.js`'s SLA calculation is used by request display, the overdue

monitor, the weekly digest *and* technician performance — four call sites, one definition, six unit tests. Had it been reimplemented inline at each call site, the four would have drifted apart the first time only one of them was updated.

- **A composite database primary key is a stronger guarantee than an application-level check.** `request_upvotes (request_id, user_id)` makes a duplicate vote structurally impossible, not merely checked-for — a lesson that generalised into treating "can the database itself prevent this?" as a real design question, not an afterthought, for every new table added afterward (see R5 in RISK_REGISTER.md).
- **`navigator.onLine` is not a reliable signal** (DEF-03/DEF-04) — a concrete instance of a broader lesson: a browser API that sounds like it answers the question you're asking ("is the user online?") sometimes only answers a narrower one ("does the network adapter have a link?"), and the gap between the two is exactly where a real defect hid.

Process

- **Small, reviewable, independently-revertible commits scale better than large ones under changing requirements.** This was tested for real, not just believed in principle: an 11-feature request delivered overnight in one sitting only stayed manageable because "commit after each feature" had already been the standing discipline for every prior feature, not something adopted specially for that one large batch.
- **Manual UAT and automated tests find genuinely different defects.** Every Transition-phase defect in DEFECT_LOG.md (DEF-03 through DEF-08) was a messaging, labelling or role-coverage issue that no unit or integration test was ever going to catch, because each one was correct *logic* behaving in a way that was merely confusing or incomplete to a real user — exactly the class of defect beta testing exists to find (see BETA_TESTING_REPORT.md §3).
- **Documenting a known limitation is more defensible than hiding one.** Both `RISK_REGISTER.md`'s two "Accepted" risks (single-instance in-memory state, a faked database in integration tests) and `ARCHITECTURE.md`'s "Known limitations" section say the same true thing in two documents on purpose — an examiner's "what would you build next?" question already has a written, honest answer rather than an improvised one.

5. What would be different with more time

Named consistently across three documents because it is the same honest answer each time it is asked: a shared store (Redis) to remove the single-instance assumption, a disposable real-database suite alongside the current faked-database integration tests, letting a student edit or cancel their own request before it is assigned, multi-campus support, and an SMS fallback for students without mobile data — see ARCHITECTURE.md §14 and VIVA_GUIDE.md §5 ("What would you build next?"). The new App Feedback channel (BETA_TESTING_REPORT.md §6) exists specifically to make that list evidence-based rather than guessed, once the system has real users.

6. Conclusion

Followed as an iterative, risk-driven process rather than a single sequential build, this project delivered every requirement in the original brief plus ten features beyond it, backed by 275 automated tests, a CI pipeline that runs on every push, and a full documentation set spanning inception artefacts through to an operator's deployment guide — all traceable, per §3, from the original vision through to a passing test. The defects that were found (DEFECT_LOG.md) were found early enough and cheaply enough to fix before submission

specifically *because* the process surfaced them in Construction and Transition rather than after — which is the entire argument for following UP in the first place.